DYNAMIC GRAPH CNN

DGCNN

Dynamic Graph CNN for Learning on Point Clouds

A visual walkthrough of the architecture, EdgeConv, and intuition

Based on the paper by Wang, Sun, Liu, Sarma, Bronstein & Solomon — MIT

MOTIVATION

Why DGCNN Was Needed

PointNet

"Look at every point on its own."

- Pushes each point through a shared MLP independently, then ONE global max-pool.
- Captures "what" the object is, but ignores how neighbouring points relate.
- No notion of local surface, edges, or corners.
- Weaker on fine structure and segmentation.

DGCNN

"Let every point talk to its neighbours."

- Connects each point to its k nearest neighbours — a local graph.
- Learns from EDGES: how a point differs from those around it.
- Captures local geometry: surfaces, edges, parts.
- Rebuilds the graph every layer — grouping by meaning, not just position.

INTUITION

The Big Idea: Learn From Relationships

A point alone says little. How it sits relative to its neighbours is what describes shape.

1

Build a graph

Connect each point to its k nearest neighbours.

2

Learn the edges

Describe how each point differs from its neighbours, not just where it is.

3

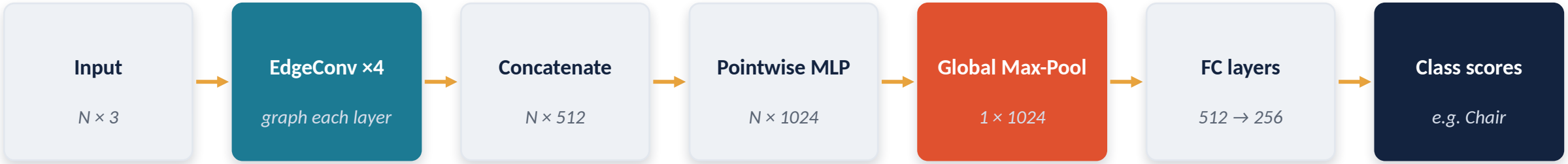
Recompute it

Rebuild the graph each layer in feature space — it becomes “dynamic.”

ARCHITECTURE

Overall Pipeline (Classification)

Skinny 3-number points are enriched layer by layer, then crushed into one fingerprint and classified.



EdgeConv stack — build geometry

decision making

N = number of points (e.g. 1024). The number after × is how many features describe each point — it grows as the network looks at more neighbourhood context, then collapses to a single vector at the global pool.

CORE BUILDING BLOCK

EdgeConv — The Repeated Block

Every EdgeConv layer does these three things for every point — then the layer is stacked.

1

Build k-NN graph

Find each point's k nearest neighbours and link them.

2

Edge features

For each neighbour: combine “who I am” with “how my neighbour differs from me.”

3

Shared MLP + max-pool

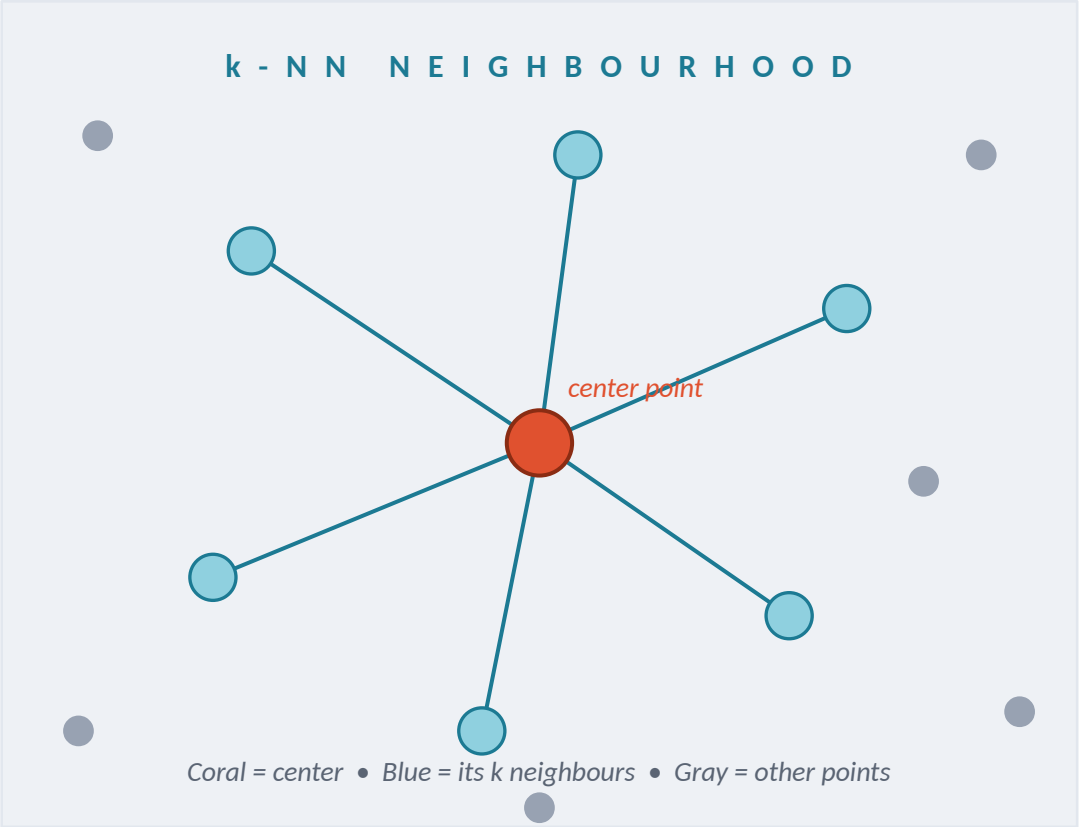
Run each edge through one mini-network, then keep the max over neighbours.

Mental model: *after EdgeConv, a point is no longer just (x, y, z) — it is a summary of its whole neighbourhood.*

Step 1 — Build the k-NN Graph

What happens

- Point clouds have no grid — no fixed “left/right” neighbour like image pixels.
- So DGCNN invents neighbours: for each point, find its k closest points (e.g. k = 20).
- Draw an edge to each — this little neighbourhood is the local region we learn from.
- Order-independent: shuffling the points gives the same graph and the same answer.

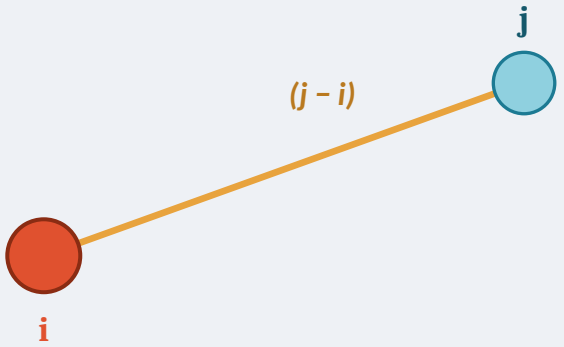


Step 2 — Build the Edge Features

What happens

- For each neighbour j, build a small packet describing the relationship to center i.
- It staples two things together: i's own features, and the difference (j − i).
- The difference is a little arrow — direction + distance — that encodes local shape.
- Keeping i's own features means the point never forgets where it globally sits.

$$edge(i \rightarrow j) = [\ \text{features of } i\ ,\ (j - i)\ ]$$



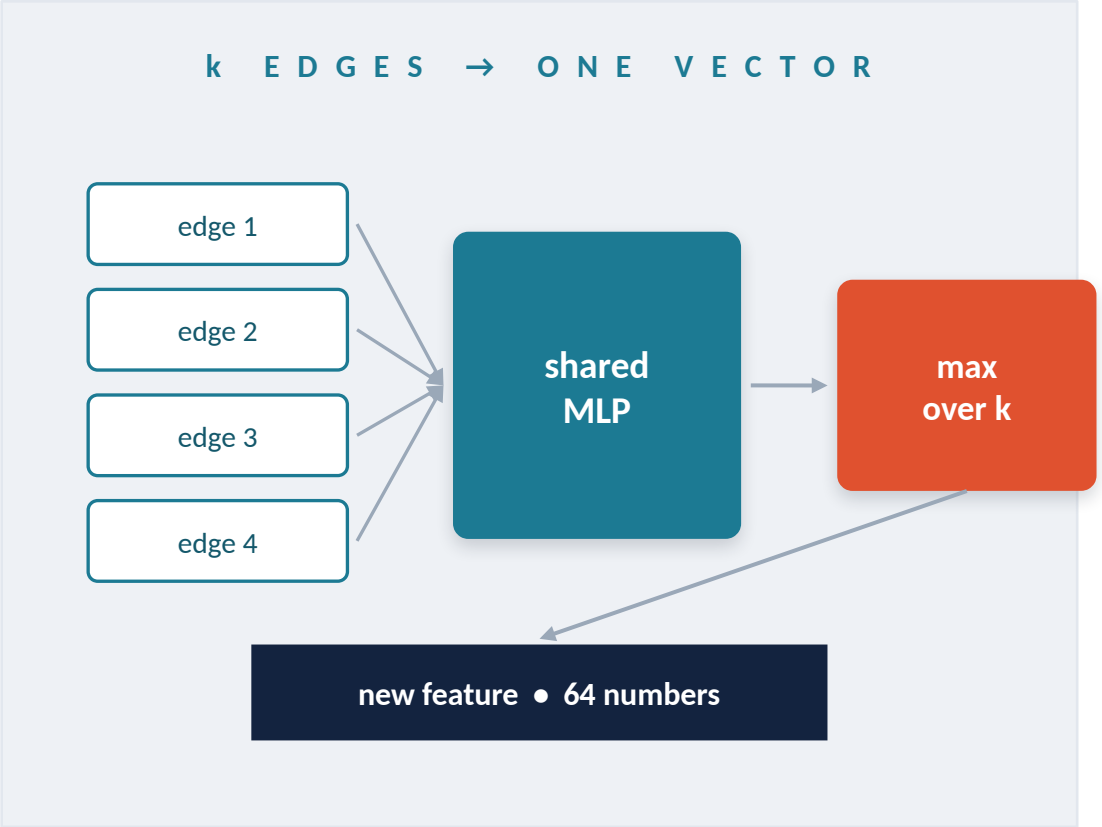
the arrow's direction & length describe the local surface

Step 3 — Shared MLP + Local Max-Pool

What happens

- Each edge runs through the SAME tiny network (shared MLP) → e.g. 64 numbers out.
- Then local max-pool: keep the biggest value in each of the 64 slots, over the k neighbours.
- That single 64-number vector becomes the point's new, richer feature.
- Shared weights + max = order doesn't matter, exactly what point clouds need.

shapes: **N × k × 64** → (max over k) → **N × 64**

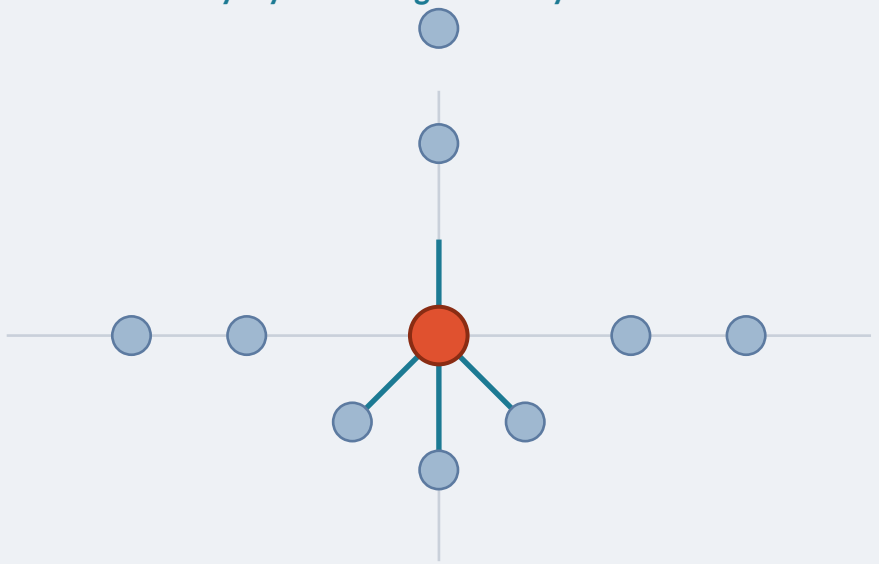


THE "DYNAMIC" PART

The Graph Is Rebuilt Every Layer

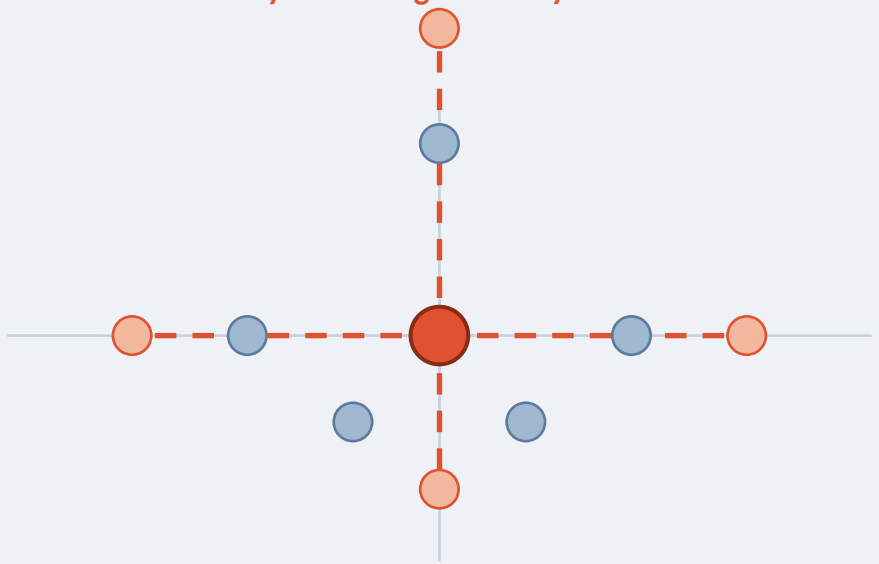
Same points — but "nearest" is recomputed from the current features, so neighbours change.

Early layer — neighbours by POSITION



links stay local

Later layer — neighbours by MEANING



links jump to far-but-similar points (both wingtips, tail)

HIERARCHY IN ACTION

What Each EdgeConv Layer Learns

EdgeConv 1

Tiny local neighbourhoods

LEARNS

edges, small curves, flat-vs-sharp surface patches

EdgeConv 2–3

Medium feature regions

LEARNS

object parts — legs, armrests, seat corners, wing sections

EdgeConv 4

Long-range, by meaning

LEARNS

symmetric & repeated structure across the whole object

PART 2

Classification Head

From one global fingerprint to a single class prediction.

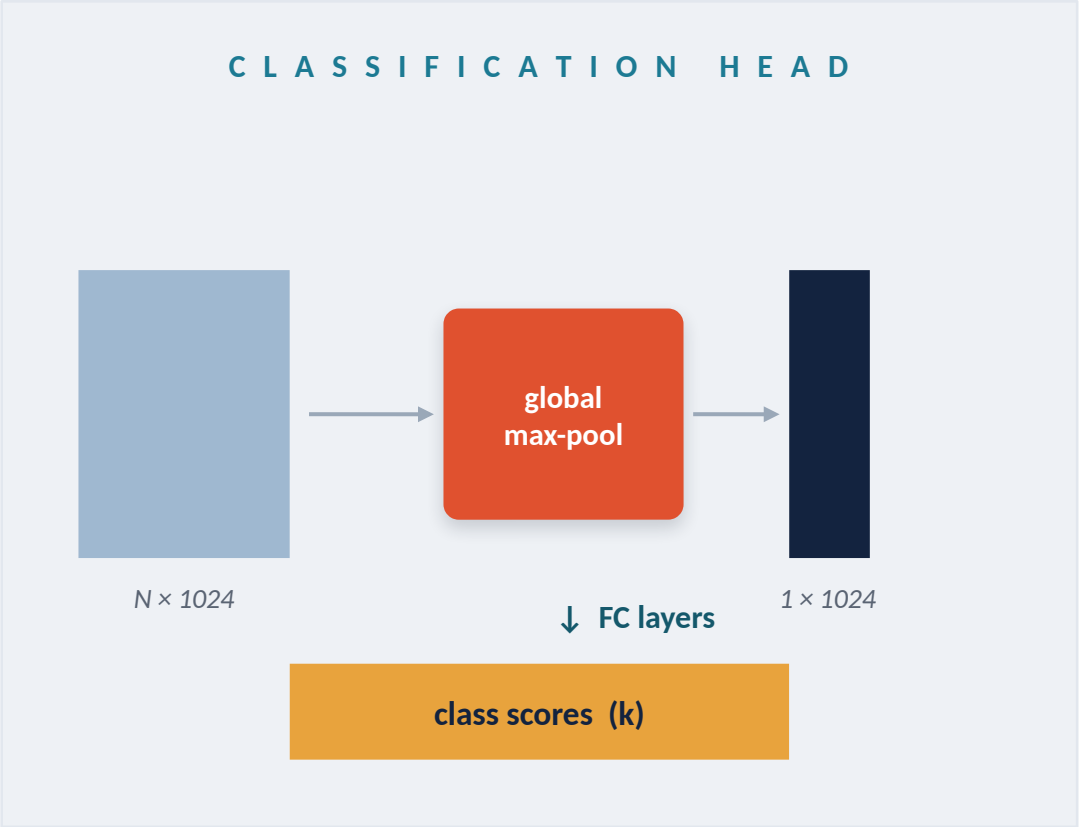
From Global Feature to Class Scores

What's the 1 × 1024 vector?

- One vector summarising the WHOLE object — not raw points anymore.
- Encodes learned patterns: “legs present”, “flat top”, “symmetry” … (not human-readable).

Fully connected layers

- A standard MLP: 1024 → 512 → 256 → k classes.
- Softmax turns scores into a probability per class.
- No graphs here — geometry was already understood by the EdgeConv stack.



Final Output: Scores → Softmax → Class

Raw scores (logits)

Class	Score
Chair	9.2
Table	1.1
Lamp	−0.5
Car	−3.0
Airplane	−5.2

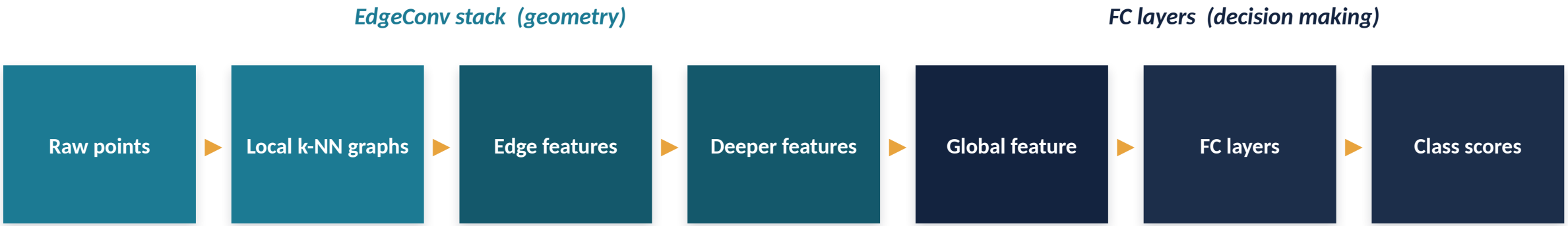
After softmax (probabilities)

Class	Probability
Chair	**96%**
Table	3%
Lamp	1%
Car	~0%
Airplane	~0%

PREDICTION **Chair** *highest softmax probability wins → final class label.*

E N D T O E N D

Complete Classification Flow



$$Final\ prediction\ =\ arg\ max\ of\ class\ scores$$

PART 3

Segmentation Head

Predict a label for every point — not just one for the whole object.

WHAT'S DIFFERENT

Segmentation vs. Classification

Classification

"What object is this?"

- One prediction for the entire cloud.
- Uses only the global feature vector.
- Example output: Chair

Segmentation

"What is EACH point?"

- One prediction per point.
- Needs global meaning AND fine local detail.
- Example output: leg / seat / back / arm …

How DGCNN Labels Every Point

No decoder needed — it reuses the per-point features it already built, plus global context.

- Stack EdgeConv layers as before — every point keeps its own feature vector (the N rows are never collapsed).
- Concatenate features from all EdgeConv layers (multi-scale: local + broad).
- Also compute the global feature, then COPY it onto every point.
- Each point now holds: local detail + global context. A shared MLP predicts its label.

Example per-point output

Point	Predicted label
p1	chair leg
p2	seat
p3	backrest
p4	armrest
p5	chair leg
…	…

RECAP

Key Takeaways

1

Graphs over grids

Point clouds have no grid; DGCNN builds a k-NN graph to create local structure.

2

EdgeConv

Per point: edge features → shared MLP → local max-pool. Order-independent.

3

Edge = self + difference

[features of i , (j − i)] captures local geometry while keeping global position.

4

Dynamic graph

Neighbours are recomputed each layer in feature space — grouping by meaning.

5

Classification

Concatenate layers → global max-pool → FC → softmax → one class.

6

Segmentation

Reuse per-point features + copied global feature → shared MLP labels every point.

C R E D I T S

Original Research Paper

Dynamic Graph CNN for Learning on Point Clouds

Yue Wang • Yongbin Sun • Ziwei Liu • Sanjay E. Sarma • Michael M. Bronstein • Justin M. Solomon

Massachusetts Institute of Technology

Published in

ACM Transactions on Graphics (TOG), 2019

Slides prepared as a visual walkthrough for educational use.